

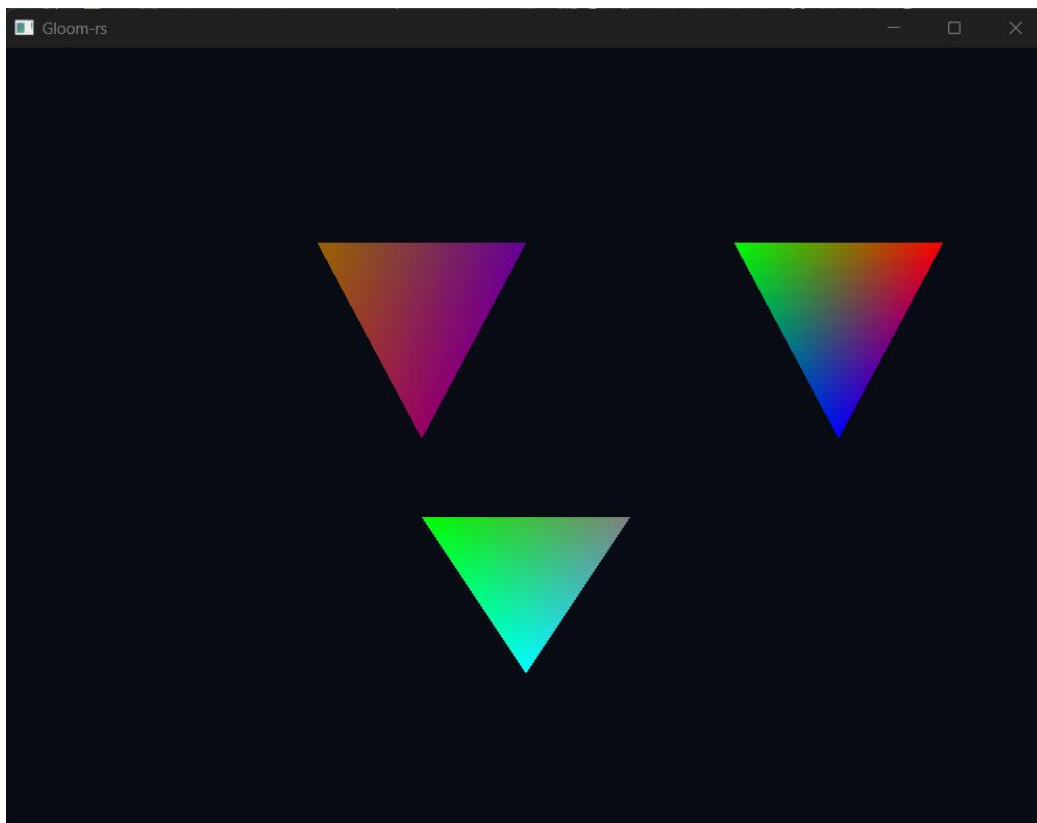
Assignment 2

Yiming Hou

Visual computing fundamentals

2025/2026

Task 1



b) Once vertices have been through the vertex shader OpenGL takes the resulting triangle and figures out which pixels on the screen are covered by this triangle. But all the values given to the vertex of each triangle (including colour attribute) are interpolated automatically across the surface and by default its interpolation is barycentric based across the screen space. In other words if vertex A is Magenta and vertex B is Purple, the pixel right in the middle of those two vertices will get a colour halfway between Magenta and Purple.

So each pixel inside the triangle will have its own values to be sent to the

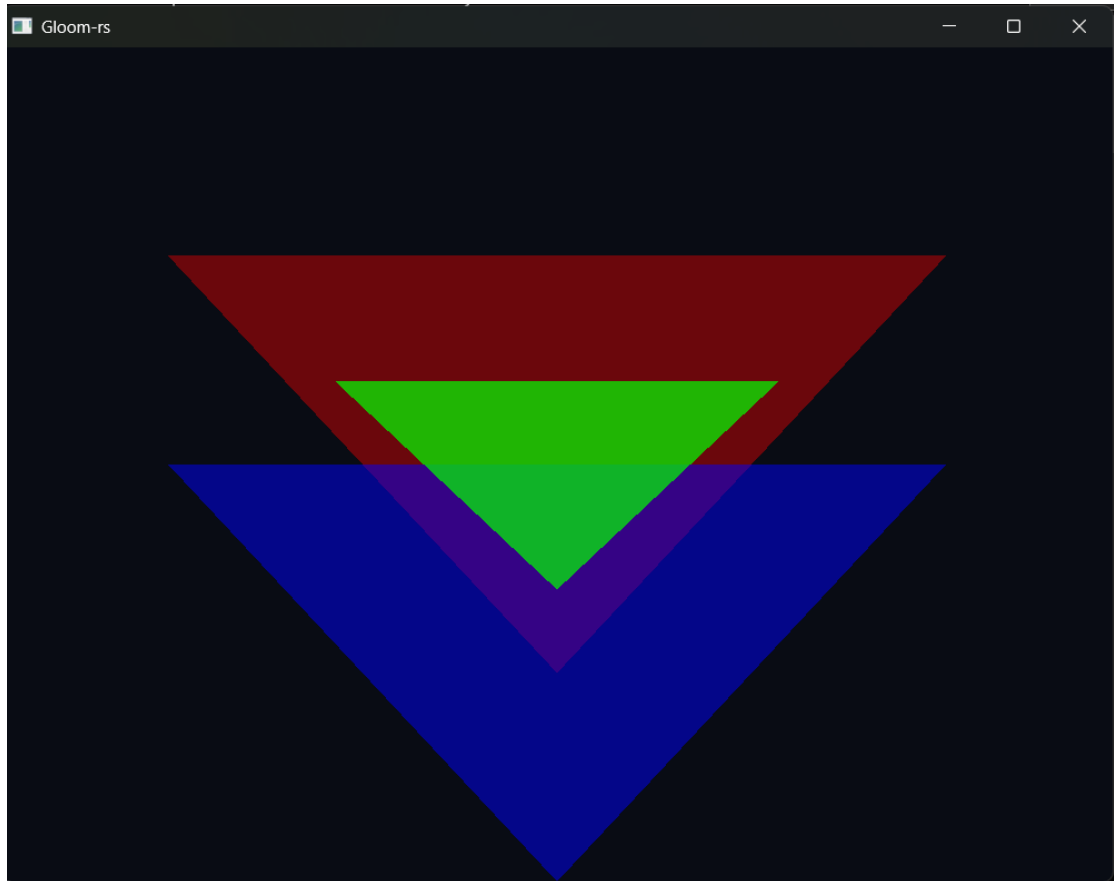
```
let colors: Vec<f32> = vec![  
  
    //triangle 1  
    1.0, 0.0, 0.0, 1.0, //v0  
    0.0, 1.0, 0.0, 1.0,  
    0.0, 0.0, 1.0, 1.0,  
  
    //2  
    0.4, 0.0, 0.6, 1.0,  
    0.6, 0.4, 0.0, 1.0,  
    0.6, 0.0, 0.4, 1.0,  
  
    //3  
    0.5, 0.5, 0.5, 1.0,  
    0.0, 1.0, 0.0, 1.0,  
    0.0, 1.0, 1.0, 1.0  
];
```

fragment shader which will use those values to give the intermediate pixel its final colour.

The resulting image is a smooth colour transition across the surface of the triangle where the colours blend harmoniously.

Task 2

a)



```
/*Three partially overlapping triangles*/
// Triangle 1 FARTHEST!
-0.7, -0.5, 0.7, // v0 bottom-left
0.7, -0.5, 0.7, // v1 bottom-right
0.0, 0.5, 0.7, // v2 top

//
-0.7,0.0,0.0,
0.7,0.0,0.0,
0.0,1.0,0.0,

// Questo non si vede
-0.4,-0.2,-0.5,
0.4,-0.2,-0.5,
0.0,0.3,-0.5

//for every vertex 4 values [RED, BLUE, GREEN, ALPHA/transparency]
let colors: Vec<f32> = vec![

    //triangle 1
    1.0, 0.0, 0.0, 0.4, //v0
    1.0, 0.0, 0.0, 0.4,
    1.0, 0.0, 0.0, 0.4,

    //2
    0.0, 0.0, 1.0, 0.5,
    0.0, 0.0, 1.0, 0.5,
    0.0, 0.0, 1.0, 0.5,

    //3
    0.0, 1.0, 0.0, 0.7,
    0.0, 1.0, 0.0, 0.7,
    0.0, 1.0, 0.0, 0.7

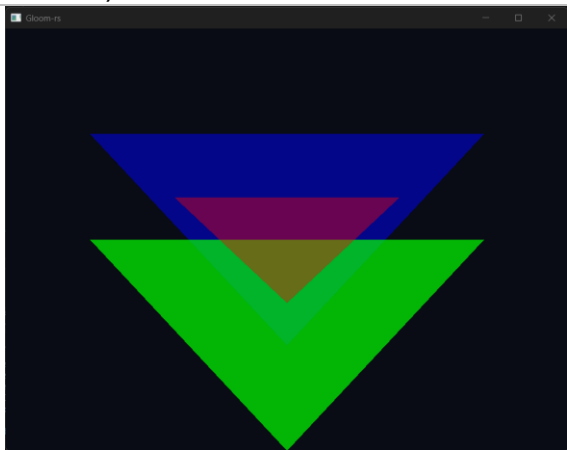
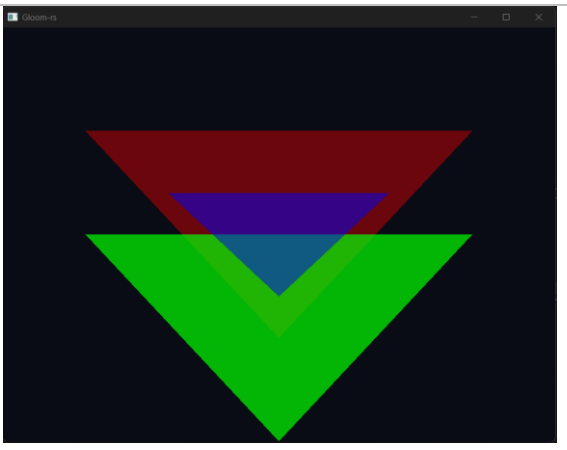
];
```

The red triangle is the farthest (higher Z coordinate) and also the faintest (lowest alpha value), the blue one is in the middle while the green one is the closest to the camera (lowest Z coordinate) while also being the least transparent among the three (highest alpha values).

The colours and their transparency are represented by 4 values in the following order [Red, Green, Blue, transparency], RGB.

b)

i. Comparison side by side

Swap to blue green red (from farthest to closest)	Swap green and blue
	
	The red behind the green triangle is barely visible due to the green triangle being placed closer to the viewer and green's high alpha value (low transparency).

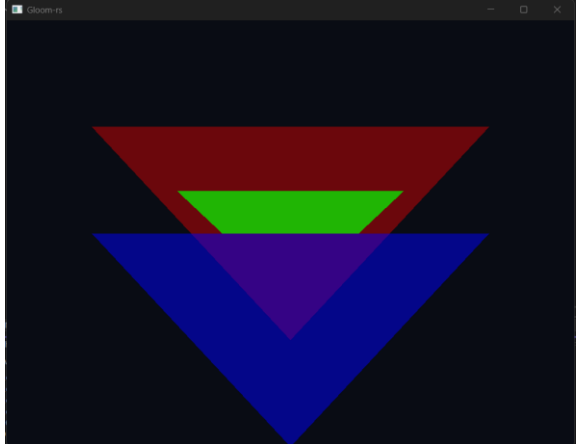
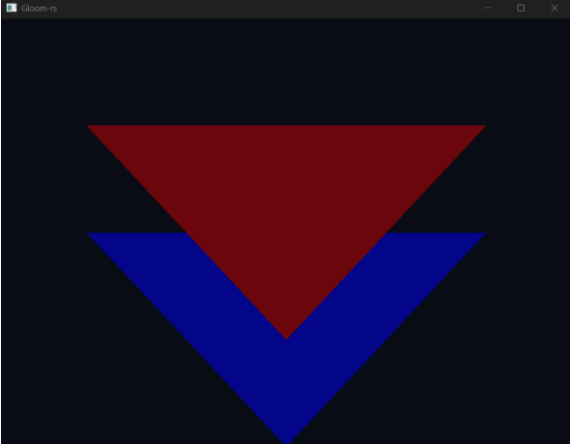
Blending is not commutative and changing colour can alter the final appearance by quite a lot, in other words the perceived blended colour depends on which colour is written before and which later, as shown by the colour equation.

$$\text{Color}_{\text{New}} = \text{Color}_{\text{Source}} \cdot \text{Alpha}_{\text{Source}} + \text{Color}_{\text{Destination}} \cdot (1 - \text{Alpha}_{\text{Source}}) \quad (1)$$

The *source* color and alpha are the color and transparency of the new pixel being drawn, respectively. The *destination* color is the color of the pixel already present at the same location in the framebuffer.

In fact, when red is on top, the area where all colours overlap resembles red the most while when blue is the closer to the viewer, blue is the most prevalent colour even though the other colours albeit faint can still be seen in the blended final colour.

ii. Comparison side by side

Swap depth 1	Swap depth 2
<pre>// Triangle 1 FARTHEST! -0.7, -0.5, 0.7, // v0 bottom-left 0.7, -0.5, 0.7, // v1 bottom-right 0.0, 0.5, 0.7, // v2 top // -0.7,0.0,0.0, 0.7,0.0,0.0, 0.0,1.0,0.0, // -0.4,-0.2,0.5, 0.4,-0.2,0.5, 0.0,0.3,0.5</pre>	<pre>-0.7, -0.5, -0.7, // v0 bottom-left 0.7, -0.5, -0.7, // v1 bottom-right 0.0, 0.5, -0.7, // v2 top // -0.7,0.0,0.0, 0.7,0.0,0.0, 0.0,1.0,0.0, // -0.4,-0.2,-0.5, 0.4,-0.2,-0.5, 0.0,0.3,-0.5 1;</pre>
	

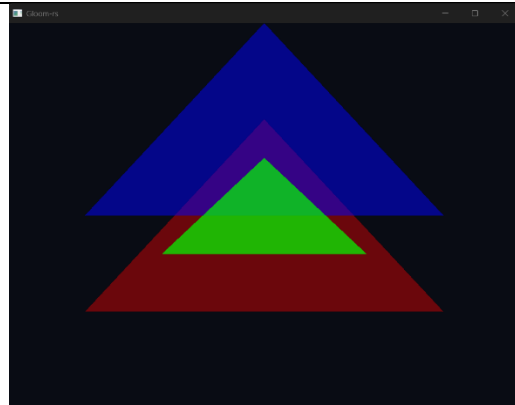
Swapping z-coordinates changes the final overlap colour completely, while swapping colours leads to blended variations. Swapping depths changes which triangle wins the depth test, so sometimes blending disappears completely (only the front-most triangle shows).

This happens because when changing the depth of the triangle without changing the order in which they get drawn and if the triangle closest to the viewer gets drawn first than the other fragments below will be discarded when the Z test is done. Therefore to blend colours it is good practice to start from the farthest element.

Task 3

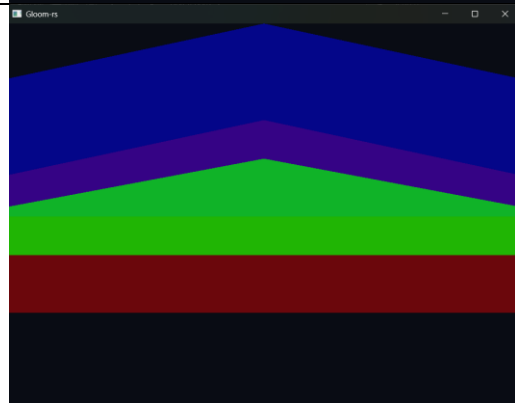
b) As seen in class, multiplying the base polygon with such matrix can induce in varied transformations across the 3D space. Considering only the position on the assignment paper (a,b,c,d,e,f) these are only 2D transformations.

On the left the resulting image and on the right the corresponding 4x4 matrix put inside simple.vert (original image is the **non-flipped** version of the overlapping triangles from Task2). GLSL is column major.



Original

On this column the matrix written inside simple.vert

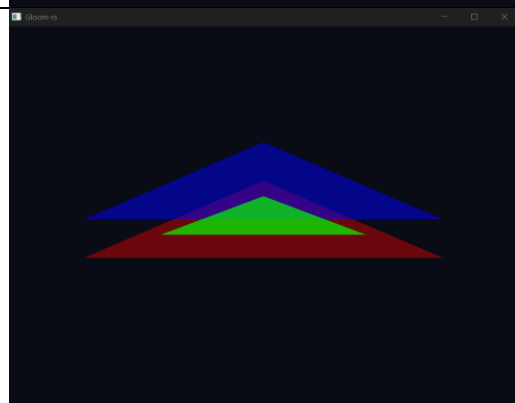


Stretched horizontally (x)

Scaling

```
myMatrix = mat4(1.0);
//initialize matrix
myMatrix[0] = vec4(5.0, 0.0, 0.0, 0.0); // column 0
myMatrix[1] = vec4(0.0, 1.0, 0.0, 0.0); // column 1
myMatrix[2] = vec4(0.0, 0.0, 1.0, 0.0); // column 2
myMatrix[3] = vec4(0.0, 0.0, 0.0, 1.0); // column 3

gl_Position = myMatrix * vec4(position, 1.0f);
```

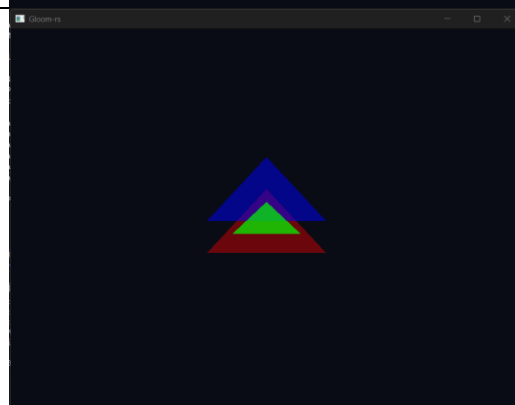


Squeezed vertically (y)

Scaling

```
myMatrix = mat4(1.0);
//initialize matrix
myMatrix[0] = vec4(1.0, 0.0, 0.0, 0.0); // column 0
myMatrix[1] = vec4(0.0, 0.4, 0.0, 0.0); // column 1
myMatrix[2] = vec4(0.0, 0.0, 1.0, 0.0); // column 2
myMatrix[3] = vec4(0.0, 0.0, 0.0, 1.0); // column 3

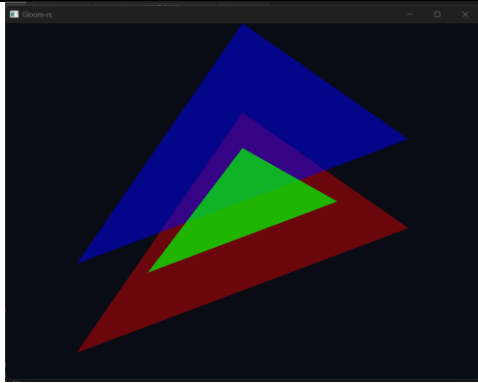
gl_Position = myMatrix * vec4(position, 1.0f);
```



Higher values of the last element in the first diagonal of the matrix makes the object look smaller while lower values makes the figure bigger

```
myMatrix = mat4(1.0);
//initialize matrix
myMatrix[0] = vec4(1.0, 0.0, 0.0, 0.0); // column 0
myMatrix[1] = vec4(0.0, 1.0, 0.0, 0.0); // column 1
myMatrix[2] = vec4(0.0, 0.0, 1.0, 0.0); // column 2
myMatrix[3] = vec4(0.0, 0.0, 0.0, 3.0); // column 3

gl_Position = myMatrix * vec4(position, 1.0f);
```



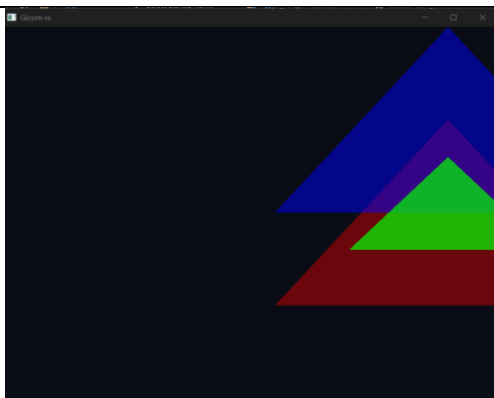
GLSL is column-major and the order is `myMatrix[column][row]`, so this configuration places 0.5 at `row=1,col=0`, below a traditional matrix.

	Col0	Col1	Col1	Col3
Row0				
Row1	0.5			
Row2				
Row3				

```
myMatrix = mat4(1.0);
//initialize matrix
myMatrix[0] = vec4(1.0, 0.5, 0.0, 0.0); // column 0
myMatrix[1] = vec4(0.0, 1.0, 0.0, 0.0); // column 1
myMatrix[2] = vec4(0.0, 0.0, 1.0, 0.0); // column 2
myMatrix[3] = vec4(0.0, 0.0, 0.0, 1.0); // column 3

gl_Position = myMatrix * vec4(position, 1.0f);
```

The closest transformation is a shear in the Y direction where the amount of vertical displacement is proportional to the X coordinate, with shear factor = 0.5.
If only “b” is modified it is another 2D shear in the X direction



	Col0	Col1	Col1	Col3
Row0				0.8
Row1				
Row2				
Row3				

```
myMatrix = mat4(1.0);
//initialize matrix
myMatrix[0] = vec4(1.0, 0.0, 0.0, 0.0); // column 0
myMatrix[1] = vec4(0.0, 1.0, 0.0, 0.0); // column 1
myMatrix[2] = vec4(0.0, 0.0, 1.0, 0.0); // column 2
myMatrix[3] = vec4(0.8, 0.0, 0.0, 1.0); // column 3

gl_Position = myMatrix * vec4(position, 1.0f);
```

Translation towards the right, in other words it is a translation in the positive direction of the X axis.
If only “f” is modified there will be a translation across the Y axis.

In a nutshell:

- The top-left 2×2 block composed by a,b,c,d applies linear transformations in the XY-plane. This can encode scaling, shearing, and reflection.
- The last column (c, f) applies translation in the X or Y axis.
- The Z axis is left untouched so this is a 2D affine transform embedded in 3D

c)

There weren't any rotation because the constraint to have a pure rotation in respect to a single point in 2D are not enforced (as seen in the matrix nearby that shows a rotation by a point of an angle theta).

$$\begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Since the variables a,b,c,d are not forced to be quartet of sines/cosines of a certain angle there is no guarantee that there will be a pure rotation. Of course, a combination of these variables that follows up those requirements will yield a rotation but since in the assignment it was asked to slowly tweak individually variable by variable it is very unlikely that a student will stumble upon a rotation as only one value will be changed each time.

Task4

Due to projection the triangles are seen from the other side (red one first).

The moving conventions follows the one suggested in the slides, it is a simulation of the point of view of a person walking in a 3 Dimensional space:

- W viewer moves closer to the shown triangles
- S viewer moves farther to the shown triangles
- A viewer moves on his left keeping the direction of his sight on the front
- D viewer moves to his right keeping the direction of his sight on the front
- Space viewer floats upwards keeping the direction of his sight on the front
- Lshift viewer goes downwards keeping the direction of his sight on the front
- Arrow keys to rotate his head and change the direction he is facing
 - NOTE: the rotation might not be evident at the starting position, it is recommended to move closer the triangles
 - NOTE: the direction of movements simulates a walking person (is relative to the WASD key directions), while facing upwards and going forward it won't float higher but only go forward following the direction the camera is facing. To float higher, one must use the Space key!

Task 5

- a) Relative movement of the camera done: now it moves following the direction the camera is facing but it is still anchored.